

Chapter 41

NLP Foundations

Clean and tokenize free text, and choose between stemming and lemmatization · build a bag-of-words and a TF-IDF representation by hand before using a library · measure similarity between documents

Approved chapter · v1

Contents

Chapter 41. NLP Foundations

Why this matters

In plain English

41.1 Text as data

41.2 Cleaning and tokenization

41.3 Bag of words and TF-IDF

41.4 Text classification

41.5 Sentiment analysis

41.6 Topic modeling

41.7 Word embeddings: the bridge to LLMs

Common mistakes and how to spot them

In the real world: the topics nobody had named

Tools

The project: classify and summarize Riverstone support tickets by topic

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 41. NLP Foundations

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: clean and tokenize free text, and choose between stemming and lemmatization · build a bag-of-words and a TF-IDF representation by hand before using a library · measure similarity between documents · classify text with Naive Bayes and logistic regression, and read a confusion matrix for more than two classes · score sentiment with a word lexicon and know when it fails · train a sentiment classifier and compare it honestly with the lexicon · find topics in unlabelled text with LDA and NMF, and judge whether they mean anything · train word embeddings and see why they're the bridge to modern language models.

Before you start: Chapter 35 (log loss, entropy), Chapter 36 (splits, pipelines), Chapter 37 (Naive Bayes, logistic regression), Chapter 38 (unsupervised learning, judging clusters without a target). This chapter treats text as another kind of feature, built on those foundations.

Time needed: 8–10 hours over one to two weeks.

Tools: Python 3 with scikit-learn, NLTK, and gensim (all free).

Practice data: 3,000 Riverstone support tickets from 2024–2025, with free-text bodies, a topic, a sentiment, and a satisfaction score, built by `companion/generate_riverstone_tickets.py`. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Chapter 1 promised that this book would eventually deal with unstructured text: emails, reviews, support tickets, free-text survey answers. That's most of the data most companies have, and almost none of the algorithms from Chapters 36–39 can touch it directly, because they all expect numbers in a table.

This chapter is the bridge. Riverstone's support inbox gets a few thousand tickets a year, and three real questions sit behind it:

- *Which team should route a new ticket to?* (classification)
- *Which customers are angry right now, so someone calls them today?* (sentiment)
- *What are people actually writing to us about, that our five official categories don't capture?* (topic discovery)

Every method here turns words into numbers first, and the quality of that conversion decides everything downstream. It's also where this book's foundations pay off directly: TF-IDF is an application of Chapter 35's dot products, text classification reuses Chapter 37's Naive Bayes and

logistic regression unchanged, and judging discovered topics uses Chapter 38's "how do you know it's real?" discipline. Nothing in this chapter is a new kind of thinking, only a new kind of input.

In plain English

Think of a support team lead skimming a stack of tickets.

First they'd scan past the "the", "and", and "please" to the words that carry meaning: *broken*, *invoice*, *late*. That's **tokenization** and **stop-word removal**. They'd notice that "delivered", "delivery", and "delivering" are really the same idea, wearing different endings. That's **stemming** or **lemmatization**.

To sort tickets into piles, they'd count which meaningful words each ticket uses, and notice that a word every ticket uses (like "order") tells them nothing, while a word only a few tickets use (like "GST") tells them a lot. That's the idea behind **TF-IDF**.

Some words carry an obvious mood: "furious", "disappointed", "thanks". Counting those is quick **sentiment analysis**, and it fails the moment someone writes something dry and factual that happens to include an angry-sounding word out of context.

Read enough tickets and patterns emerge that nobody wrote down as an official category: maybe there's a cluster of tickets that are really about *packaging*, cutting across "delivery" and "defect". Finding those patterns without being told what to look for is **topic modeling**.

And if you've ever noticed that a customer who says "crate" often also says "broken" or "cracked", never "refund" or "invoice", you've done in your head what a **word embedding** does with arithmetic: it learns which words keep company with which.

41.1 Text as data

The tickets

```
import re
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

tickets = pd.read_csv("../tickets/tickets.csv")
print(
    f"{len(tickets):,} tickets, {tickets['created_at'].min()} "
    f"to {tickets['created_at'].max()}"
)
print(tickets["topic"].value_counts().to_string())
print(tickets["sentiment"].value_counts().to_string())
print(
    f"\nbody length in characters: mean {tickets['body'].str.len().mean():.0f}, "
    f"min {tickets['body'].str.len().min()}, max {tickets['body'].str.len().max()}"
)
print("\none ticket:")
print(tickets.loc[0, ["subject", "body", "topic", "sentiment"]].to_string())
```

```
3,000 tickets, 2024-01-01 to 2025-12-30
topic
Delivery      892
Product Defect 625
Billing        560
General Enquiry 514
Order Change   409
sentiment
neutral      1467
frustrated   988
positive     545

body length in characters: mean 120, min 73, max 230

one ticket:
subject      Billing issue
body         Dear team, could you clarify the invoice for o...
topic        Billing
sentiment     neutral
```

Riverstone's tickets look like real support email: short, sometimes typo-ridden, occasionally polite ("kindly do the needful"), occasionally furious. Five official topics and three sentiment labels were assigned when each ticket was generated, which lets this chapter check machine answers against a known truth, something you rarely get with real support data.

A caution about this chapter's data. These tickets come from a template generator: a handful of sentence patterns per topic and sentiment, filled in with random products and numbers. That makes the topics far more distinctive than typical human writing, and you'll see classification scores in this chapter that would be suspicious on a real inbox. Section 41.2's watch-out box and the exercises come back to this; treat the *methods* as the lesson, not the exact scores.

Why "just read them" doesn't scale

At 3,000 tickets a year, reading every one is possible; at 30,000 it isn't, and a human reader is also inconsistent (two people label the same ticket differently) and slow to notice a trend spread across weeks. The rest of the chapter turns text into something a computer, and a consistent process, can act on.

41.2 Cleaning and tokenization

Tokenization splits text into units, usually words. NLTK's tokenizer knows the difference between a sentence-ending period and a decimal point, and separates punctuation from words:

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

raw = tickets.loc[0, "body"]
lower = raw.lower()
tokens = word_tokenize(lower)
print("raw:      ", raw)
print("lowercase:", lower)
print("tokens:   ", tokens)

words_only = [t for t in tokens if t.isalpha()]
print("\nwords only (punctuation and numbers dropped):", words_only)

stop_words = set(stopwords.words("english"))
kept = [t for t in words_only if t not in stop_words]
print(f"after removing {len(words_only) - len(kept)} stop words:", kept)
```

```
raw:      Dear team, could you clarify the invoice for order #108347? The amount Rs 2570 does not
match what we discussed with the sales rep.
lowercase: dear team, could you clarify the invoice for order #108347? the amount rs 2570 does not
match what we discussed with the sales rep.
tokens:   ['dear', 'team', ',', 'could', 'you', 'clarify', 'the', 'invoice', 'for', 'order', '#',
'108347', '?', 'the', 'amount', 'rs', '2570', 'does', 'not', 'match', 'what', 'we', 'discussed',
'with', 'the', 'sales', 'rep', '.']

words only (punctuation and numbers dropped): ['dear', 'team', 'could', 'you', 'clarify', 'the',
'invoice', 'for', 'order', 'the', 'amount', 'rs', 'does', 'not', 'match', 'what', 'we',
'discussed', 'with', 'the', 'sales', 'rep']
after removing 10 stop words: ['dear', 'team', 'could', 'clarify', 'invoice', 'order', 'amount',
'rs', 'match', 'discussed', 'sales', 'rep']
```

Reading it. `word_tokenize` split `#108347` into `#` and `108347`, and kept `sales` and `rep` apart correctly. Filtering to alphabetic tokens drops numbers and punctuation, which is usually right for topic and sentiment work and usually wrong if the numbers matter (an amount, a date). **Stop words** (*the, is, and, we*) carry grammar, not meaning, and removing them shrinks the vocabulary without losing much signal for the tasks in this chapter.

Stemming vs lemmatization

Both reduce words to a common form, so "deliver", "delivered", and "delivering" count as one feature instead of three. They do it differently:

- **Stemming** (Porter's algorithm) chops suffixes with fixed rules, fast and crude. It doesn't know grammar, so it can produce non-words.
- **Lemmatization** looks up the dictionary base form (the **lemma**), using part of speech to decide (*better* as an adjective has no simpler form; as nothing in particular, WordNet leaves it alone).

```
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
examples = [
    "delivered",
    "delivery",
    "delivering",
    "crates",
    "broken",
    "running",
    "better",
]
print(f'{"word":<12}{ "stem":<12}{ "lemma (verb)":<14}{ "lemma (noun)":<14}')
for word in examples:
    print(
        f'{"word":<12}{stemmer.stem(word):<12}{lemmatizer.lemmatize(word, pos='v'):<14}'
        f'{"lemma (verb)":<14}{lemmatizer.lemmatize(word, pos='n'):<14}'
    )
```

word	stem	lemma (verb)	lemma (noun)
delivered	deliv	deliver	delivered
delivery	deliveri	delivery	delivery
delivering	deliv	deliver	delivering
crates	crate	crate	crate
broken	broken	break	broken
running	run	run	running
better	better	better	better

Reading it. Stemming turns *delivery* into *deliveri*, not a real word, but a consistent bucket that groups with *delivered* and *delivering* just as well. Lemmatization needs to be told the part of speech to work properly: told "these are verbs", it correctly maps *broken* to *break* and *running* to *run*; without that hint it would leave many words unchanged. For search and topic modeling, stemming's crudeness rarely matters and it's faster; for anything a human will read afterward (a word cloud, a report), lemmatization looks less broken.

Watch out: cleaning choices are invisible in the output but change everything downstream.

Whether you keep numbers, remove stop words, stem, or lemmatize is a modeling decision like any hyperparameter, and different choices can move a classifier's accuracy by several points. Try more than one and compare (exercise 6), the same discipline as Chapter 36's feature engineering.

41.3 Bag of words and TF-IDF

Bag of words, by hand

The simplest representation: for a fixed vocabulary, count how many times each word appears in each document, ignoring order entirely (hence "bag").

```
mini = [
    "the crate arrived broken",
    "the crate arrived on time",
    "please send a replacement crate",
]
vocab = sorted(set(word for doc in mini for word in doc.split()))
print("vocabulary:", vocab)
bow = pd.DataFrame(
    [[doc.split().count(w) for w in vocab] for doc in mini], columns=vocab
)
print(bow.to_string())
```

```
vocabulary: ['a', 'arrived', 'broken', 'crate', 'on', 'please', 'replacement', 'send', 'the', 'time']
   a  arrived  broken  crate  on  please  replacement  send  the  time
0  0         1       1     1   0        0           0    0    1     0
1  0         1       0     1   1        0           0    0    1     1
2  1         0       0     1   0        1           1    1    0     0
```

Reading it. Each row is a document, each column a vocabulary word, each cell a count. "Crate" appears in all three; "arrived" in the first two; "the" in the first two, dropped in the third only because that sentence happens not to use it. Order is completely gone: "the crate arrived broken" and "broken arrived the crate" would produce an identical row. That's the bag-of-words trade: simple and fast, blind to word order and negation ("not broken" looks like "broken" plus a stop word).

TF-IDF, by hand

Raw counts overweight common words. **TF-IDF** (term frequency-inverse document frequency) discounts a word by how many documents contain it:

$$\text{tf-idf}(\text{word}, \text{doc}) = \text{tf}(\text{word}, \text{doc}) \times \text{idf}(\text{word}), \text{ where } \text{idf}(\text{word}) = \log((1 + N) \div (1 + \text{documents containing word})) + 1$$

(The "+1"s and the trailing "+1" are scikit-learn's smoothing, so no word gets a zero weight and no division fails on a word that's in every document.)

```

n_docs = len(mini)

def idf(word):
    docs_with_word = sum(word in doc.split() for doc in mini)
    return (
        np.log((1 + n_docs) / (1 + docs_with_word)) + 1
    ) # scikit-learn's smoothed formula

for word in ["crate", "broken", "arrived", "the"]:
    tf = mini[0].split().count(word)
    print(
        f"{word:<8} tf(doc 1)={tf}    idf={idf(word):.3f} "
        f"    tf-idf(doc 1)={tf * idf(word):.3f}"
    )

```

crate	tf(doc 1)=1	idf=1.000	tf-idf(doc 1)=1.000
broken	tf(doc 1)=1	idf=1.693	tf-idf(doc 1)=1.693
arrived	tf(doc 1)=1	idf=1.288	tf-idf(doc 1)=1.288
the	tf(doc 1)=1	idf=1.288	tf-idf(doc 1)=1.288

Reading it. "Crate" appears in all three mini-documents, so its idf is a bare 1.0, the smallest possible: common within this collection, not distinctive. "Broken" appears in only one, so it gets the highest idf and the highest tf-idf score in that document: a word that appears rarely across the collection but stands out where it does appear is exactly what TF-IDF is built to find.

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
matrix = vectorizer.fit_transform(mini)
print("vocabulary:", vectorizer.get_feature_names_out())
scores = pd.DataFrame(matrix.toarray(), columns=vectorizer.get_feature_names_out())
print(scores.round(3).to_string())
print(
    f"\nrow norm (scikit-learn normalizes each row to length 1): "
    f"{np.linalg.norm(scores.iloc[0]):.3f}"
)

```

```

vocabulary: ['arrived' 'broken' 'crate' 'on' 'please' 'replacement' 'send' 'the'
'time']
   arrived  broken  crate   on  please  replacement  send   the  time
0    0.480   0.632  0.373  0.000   0.000         0.000  0.000  0.480  0.000
1    0.406   0.000  0.315  0.534   0.000         0.000  0.000  0.406  0.534
2    0.000   0.000  0.323  0.000   0.546         0.546  0.546  0.000  0.000

row norm (scikit-learn normalizes each row to length 1): 1.000

```

How it works: scikit-learn's `TfidfVectorizer` does the counting, the idf weighting, and one more step: it rescales each document's row to length 1 (its Euclidean norm), so that a long document and a short one on the same topic get comparable vectors. That's Chapter 35's vector normalization, applied to text.

Documents as vectors, again

Once every ticket is a TF-IDF vector, Chapter 35's cosine similarity finds the tickets that read most alike:

```
from sklearn.metrics.pairwise import cosine_similarity

full_tfidf = TfidfVectorizer(stop_words="english", max_features=3000, min_df=2)
X_all = full_tfidf.fit_transform(tickets["body"])
sim = cosine_similarity(X_all[0], X_all)[0]
nearest = np.argsort(-sim)[1:4]
print(f"ticket 0: {tickets.loc[0, 'topic']} / {tickets.loc[0, 'sentiment']}")
print(tickets.loc[0, "body"])
for i in nearest:
    print(
        f"\nsimilarity {sim[i]:.3f} -- ticket {i} "
        f"{tickets.loc[i, 'topic']} / {tickets.loc[i, 'sentiment']}:"
    )
    print(tickets.loc[i, "body"])
```

```
ticket 0: Billing / neutral
Dear team, could you clarify the invoice for order #108347? The amount Rs 2570 does not match what
we discussed with the sales rep.

similarity 1.000 -- ticket 2991 (Billing / neutral):
Dear team, could you clarify the invoice for order #103286? The amount Rs 14140 does not match what
we discussed with the sales rep.

similarity 1.000 -- ticket 64 (Billing / neutral):
Dear team, could you clarify the invoice for order #103017? The amount Rs 20960 does not match what
we discussed with the sales rep.

similarity 1.000 -- ticket 2817 (Billing / neutral):
Dear team, could you clarify the invoice for order #110927? The amount Rs 10920 does not match what
we discussed with the sales rep.
```

Reading it. The three most similar tickets to ticket 0 score a cosine similarity of essentially 1.000: the same sentence template with a different order number and amount swapped in. That's expected given how this data was built, and it makes the exact point TF-IDF is designed around: order numbers and amounts get almost no weight (they're unique to each document, so their idf is high but their tf is 1, and more importantly the *shared* words, "clarify", "invoice", "amount", "match", "sales", "rep", dominate the similarity). On real support tickets, this same calculation is what powers "find similar past tickets" and duplicate-ticket detection.

41.4 Text classification

Setting up, and a caution about scores

Split by a stratified random split (there's no time-ordering concern here, unlike Chapter 36's leads, since each ticket is independent):

```

from sklearn.model_selection import train_test_split

train, test = train_test_split(
    tickets, test_size=0.25, random_state=41, stratify=tickets["topic"]
)
print(f"{len(train):,} training tickets, {len(test):,} test tickets")
print((train["topic"].value_counts(normalize=True) * 100).round(1).to_string())

```

```

2,250 training tickets, 750 test tickets
topic
Delivery      29.7
Product Defect 20.8
Billing        18.7
General Enquiry 17.1
Order Change   13.6

```

Train Naive Bayes and logistic regression on TF-IDF features to predict the ticket's topic, exactly the algorithms from Chapter 37, now fed text instead of a feature table:

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

def topic_pipeline(model):
    return Pipeline(
        [
            (
                "tfidf",
                TfidfVectorizer(
                    stop_words="english",
                    max_features=3000,
                    ngram_range=(1, 2),
                    min_df=2,
                ),
            ),
            ("model", model),
        ]
    )

nb_topic = topic_pipeline(MultinomialNB()).fit(train["body"], train["topic"])
lr_topic = topic_pipeline(LogisticRegression(max_iter=1000)).fit(
    train["body"], train["topic"]
)
for name, model in [("Naive Bayes", nb_topic), ("logistic regression", lr_topic)]:
    pred = model.predict(test["body"])
    print(
        f"{name:<22} accuracy {(pred == test['topic']).mean():.3f}  "
        f"macro F1 {f1_score(test['topic'], pred, average='macro'):.3f}"
    )
print()
print(classification_report(test["topic"], lr_topic.predict(test["body"])))

```

Naive Bayes	accuracy 0.995	macro F1 0.995		
logistic regression	accuracy 0.995	macro F1 0.995		
	precision	recall	f1-score	support
Billing	1.00	0.99	0.99	140
Delivery	0.98	1.00	0.99	223
General Enquiry	1.00	1.00	1.00	129
Order Change	1.00	0.99	1.00	102
Product Defect	1.00	0.99	1.00	156
accuracy			0.99	750
macro avg	1.00	0.99	1.00	750
weighted avg	0.99	0.99	0.99	750

Reading it. Both models score above 99% accuracy and macro F1. That is a suspiciously good result, and section 41.1's caution explains why: this generator uses only two or three sentence templates per topic, so the vocabulary is nearly a fingerprint for the category. **Real support tickets, written by hundreds of different people in their own words, would not classify this cleanly;** a well-tuned production system on genuine free text typically lands in the 80s or low 90s for accuracy on a five-way problem like this. Treat this score as a demonstration that the method works, not as a benchmark to expect elsewhere.

The confusion matrix, and one thing it can't explain

```
from sklearn.metrics import confusion_matrix

labels = sorted(tickets["topic"].unique())
cm = confusion_matrix(test["topic"], lr_topic.predict(test["body"]), labels=labels)
print(pd.DataFrame(cm, index=labels, columns=labels).to_string())
mistakes = test[lr_topic.predict(test["body"]) != test["topic"]]
print(f"\n{len(mistakes)} misclassified tickets out of {len(test)}")
if len(mistakes):
    row = mistakes.iloc[0]
    print(
        f"example: true={row['topic']}, predicted={lr_topic.predict([row['body']])[0]}"
    )
    print(row["body"])
```

	Billing	Delivery	General Enquiry	Order Change	Product Defect
Billing	138	2	0	0	0
Delivery	0	223	0	0	0
General Enquiry	0	0	129	0	0
Order Change	0	1	0	101	0
Product Defect	0	1	0	0	155

```
4 misclassified tickets out of 750
example: true=Billing, predicted=Delivery
The you was fine but the delivery was late and the invoice amount looks off, please check.
```

Reading it. Only 4 of 750 tickets are misclassified. The one error shown is genuinely hard: a "mixed" ticket, deliberately planted in the generator, that mentions a late delivery and an invoice problem in the same message and never uses the words that anchor "Product Defect" clearly. Even humans disagree on tickets like this; a classification system needs a rule for them (route to the topic mentioned first, or to a "mixed issue" queue) rather than pretending every ticket has one true label.

Checking the score isn't an accident of IDs

```
import re as _re

digits = _re.compile(r"\d+")
placeholder_train = train["body"].str.replace(digits, "0", regex=True)
placeholder_test = test["body"].str.replace(digits, "0", regex=True)
lr_no_ids = topic_pipeline(LogisticRegression(max_iter=1000)).fit(
    placeholder_train, train["topic"]
)
pred_no_ids = lr_no_ids.predict(placeholder_test)
print(
    f"with order numbers replaced by a placeholder: "
    f"accuracy {(pred_no_ids == test['topic']).mean():.3f}"
)
print(
    f"original (numbers kept):"
    f"accuracy {(lr_topic.predict(test['body']) == test['topic']).mean():.3f}"
)
```

```
with order numbers replaced by a placeholder: accuracy 0.995
original (numbers kept):                        accuracy 0.995
```

Reading it. Replacing every digit with a placeholder before training changes accuracy not at all. That's reassuring: the classifier isn't secretly memorizing which order numbers happened to appear in which category (they couldn't, since each number appears in only one ticket in the whole dataset); it's genuinely using the topic-specific vocabulary. This is the text-classification version of Chapter 36's leakage check: before trusting a score, ask what the model could be secretly keying on.

When Naive Bayes and logistic regression are the right choice for text: both are fast to train even with a huge vocabulary, both work well on sparse TF-IDF features, and both are still standard first choices for text classification at moderate scale. Naive Bayes is faster and simpler to explain; logistic regression usually edges it on accuracy and gives probabilities that are easier to calibrate (Chapter 39).

41.5 Sentiment analysis

The lexicon approach

A **sentiment lexicon** is a dictionary of words with a polarity score, built by human annotation once and reused everywhere. **VADER** (Valence Aware Dictionary and sEntiment Reasoner) is tuned for informal text and handles negation ("not good") and intensifiers ("very good") with simple rules, needing no training data at all.

```

from nltk.sentiment import SentimentIntensityAnalyzer

nltk.download("vader_lexicon", quiet=True)
sia = SentimentIntensityAnalyzer()
tickets["vader_score"] = tickets["body"].apply(
    lambda b: sia.polarity_scores(b)["compound"]
)
print(
    tickets.groupby("sentiment")["vader_score"]
    .agg(["mean", "std"])
    .round(3)
    .to_string()
)

lexicon_pred = pd.cut(
    tickets["vader_score"],
    bins=[-2, -0.05, 0.05, 2],
    labels=["frustrated", "neutral", "positive"],
)
agreement = (lexicon_pred.astype(str) == tickets["sentiment"]).mean()
print(f"\nlexicon-rule accuracy against the labeled sentiment: {agreement:.1%}")
print(
    pd.crosstab(
        tickets["sentiment"],
        lexicon_pred,
        rownames=["labeled"],
        colnames=["lexicon says"],
    ).to_string()
)

```

```

          mean    std
sentiment
frustrated -0.332  0.340
neutral    0.217  0.279
positive   0.644  0.159

lexicon-rule accuracy against the labeled sentiment: 62.5%
lexicon says  frustrated  neutral  positive
labeled
frustrated           762         88       138
neutral              76        579       812
positive              0         12       533

```

Reading it. The average VADER score does separate the three labeled sentiments in the right direction (−0.33, 0.22, 0.64), but turning that continuous score into a hard yes/no/neutral call gets it right only **62.5%** of the time. The confusion table shows why: 812 tickets labeled "neutral" get called "positive" by the lexicon, more than the neutral tickets it gets right.

```

worst = (tickets["sentiment"] == "neutral") & (lexicon_pred == "positive")
example = tickets[worst].iloc[0]
print(
    f"labeled: {example['sentiment']}    VADER score: {example['vader_score']:.2f} -> "
    f"lexicon says: positive"
)
print(example["body"])

```

```

labeled: neutral    VADER score: 0.38 -> lexicon says: positive
Dear team, could you clarify the invoice for order #108347? The amount Rs 2570 does not match what
we discussed with the sales rep.

```

Reading it. A routine billing query ("could you clarify the invoice... does not match...") scores +0.38, "positive", because VADER doesn't know that a mismatched invoice is a problem; it just doesn't contain any word its dictionary flags as negative, and dry, polite business language reads as mildly positive by VADER's general-purpose calibration. **A lexicon has no idea what your business considers bad news.** It also can't learn from your data.

A trained classifier

```
def sentiment_pipeline(model):
    return Pipeline(
        [
            (
                "tfidf",
                TfidfVectorizer(
                    stop_words="english",
                    max_features=3000,
                    ngram_range=(1, 2),
                    min_df=2,
                ),
            ),
            ("model", model),
        ]
    )

lr_sent = sentiment_pipeline(LogisticRegression(max_iter=1000)).fit(
    train["body"], train["sentiment"]
)
pred_sent = lr_sent.predict(test["body"])
print(
    f"trained classifier accuracy: {(pred_sent == test['sentiment']).mean():.3f}    "
    f"macro F1 {f1_score(test['sentiment'], pred_sent, average='macro'):.3f}"
)
print(
    f"lexicon rule accuracy (same test rows): "
    f"{(lexicon_pred.loc[test.index].astype(str) == test['sentiment']).mean():.3f}"
)
```

```
trained classifier accuracy: 0.999    macro F1 0.999
lexicon rule accuracy (same test rows): 0.629
```

Reading it. A logistic regression trained directly on labeled tickets reaches 99.9% accuracy, again inflated by the templated data (section 41.1), but the *comparison* is the honest and durable lesson: **a model trained on your own labels beats a generic lexicon by a wide margin**, because it learns which words and phrases mean trouble specifically in your context ("does not match", "twice", "unacceptable") rather than relying on a dictionary built for restaurant reviews and tweets.

When to use which. A lexicon needs zero labeled data and works today; use it for a quick first pass or when you truly have no training examples. The moment you have even a few hundred labeled examples of your own text, a trained classifier is worth building, and it will keep improving as you label more.

41.6 Topic modeling

Finding structure with no labels

Topic modeling discovers groups of words that tend to co-occur, without being told any categories, the text equivalent of Chapter 38's clustering. **Latent Dirichlet Allocation (LDA)** treats each document as a mixture of topics and each topic as a distribution over words, fitted so the documents look as likely as possible under that story. **NMF** (non-negative matrix factorization) finds the same kind of structure by decomposing the TF-IDF matrix directly, with a similar effect and a different mechanism.

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

topic_vectorizer = CountVectorizer(
    stop_words="english", max_features=1000, min_df=5, token_pattern=r"\b[a-z]{3,}\b"
)
counts = topic_vectorizer.fit_transform(tickets["body"].str.lower())
lda = LatentDirichletAllocation(n_components=5, random_state=41, max_iter=20).fit(
    counts
)
words = topic_vectorizer.get_feature_names_out()
for i, component in enumerate(lda.components_):
    top_words = [words[j] for j in component.argsort()[-8:][::-1]]
    print(f"LDA topic {i}: {' '.join(top_words)}")

tickets["lda_topic"] = lda.transform(counts).argmax(axis=1)
print()
print(pd.crosstab(tickets["topic"], tickets["lda_topic"]).to_string())
```

LDA topic 0: order, team, immediately, invoice, dear, gst, copy, accounts
LDA topic 1: order, days, update, check, address, wanted, status, share
LDA topic 2: order, days, just, team, dear, time, called, ago
LDA topic 3: order, invoice, does, arrived, team, dear, rep, sales
LDA topic 4: order, units, website, soon, range, place, really, team

lda_topic	0	1	2	3	4
topic					
Billing	323	4	0	233	0
Delivery	0	281	415	102	94
General Enquiry	0	2	299	0	213
Order Change	0	164	64	0	181
Product Defect	164	2	242	217	0

```
from sklearn.decomposition import NMF

nmf_vectorizer = TfidfVectorizer(
    stop_words="english", max_features=1000, min_df=5, token_pattern=r"\b[a-z]{3,}\b"
)
tfidf_counts = nmf_vectorizer.fit_transform(tickets["body"].str.lower())
nmf = NMF(n_components=5, random_state=41, max_iter=400).fit(tfidf_counts)
nmf_words = nmf_vectorizer.get_feature_names_out()
for i, component in enumerate(nmf.components_):
    top_words = [nmf_words[j] for j in component.argsort()[-8:][::-1]]
    print(f"NMF topic {i}: {' '.join(top_words)}")

tickets["nmf_topic"] = nmf.transform(tfidf_counts).argmax(axis=1)
print()
print(pd.crosstab(tickets["topic"], tickets["nmf_topic"]).to_string())
```

NMF topic 0: order, units, disappointed, dear, bulk, time, team, ago
 NMF topic 1: status, dispatch, check, share, update, wanted, days, order
 NMF topic 2: tracking, page, updated, checking, reach, just, days, order
 NMF topic 3: invoice, gst, team, copy, accounts, requesting, shows, wrong
 NMF topic 4: called, twice, acceptable, delivered, account, deducted, times, charged

nmf_topic	0	1	2	3	4
topic					
Billing	0	4	0	439	117
Delivery	257	283	230	0	122
General Enquiry	397	2	3	112	0
Order Change	390	18	0	1	0
Product Defect	560	61	0	4	0

LDA topic 0	LDA topic 1	LDA topic 2	LDA topic 3	LDA topic 4
order	order	order	order	order
team	days	days	invoice	units
immediately	update	just	does	website
invoice	check	team	arrived	soon
dear	address	dear	team	range
gst	wanted	time	dear	place
copy	status	called	rep	really
accounts	share	ago	sales	team

Discovered LDA topic vs. real category (row share, %)

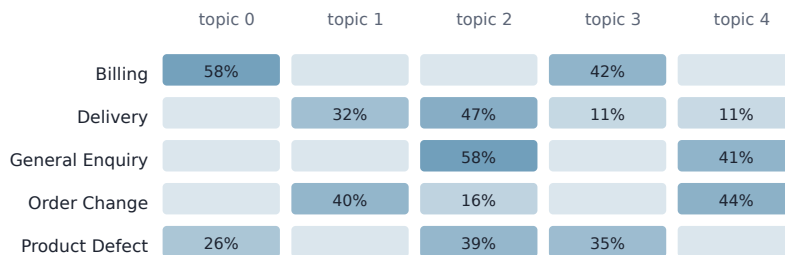


Figure 41.1 – LDA and NMF topics, and how each maps onto the five real categories. Neither method rediscovers the official five; both find something else: written tone and specific phrasing.

Reading it, honestly, as Chapter 38 taught. Neither method's five topics line up with the five real categories. NMF topic 4 ("called, twice, acceptable, delivered, deducted, charged") is angry Billing and angry Delivery tickets bundled together by their frustrated vocabulary, not by subject. LDA topic 2 ("days, just, team, time, called, ago, tracking") is mostly Delivery, but with a third of Product Defect tickets mixed in, wherever those tickets happen to mention days and time. **The topics found are real** (there genuinely is a cluster of "curt, angry, wants action now" language cutting across categories), **but they aren't the five categories a support manager already has**, and no amount of tuning would necessarily make them line up, because nothing told the algorithm those five categories exist. This is the exact caution from Chapter 38, section 38.2: an internal, algorithm-driven grouping is not automatically the grouping a business already uses.

What topic modeling is actually good for, given this: discovering categories nobody has named yet ("a cluster of angry, urgent language" is itself useful information for staffing an escalations queue), a first pass on genuinely unlabelled data before anyone builds a taxonomy, and a sanity check on whether your existing categories capture what people are actually writing about. It's a poor substitute for labeled classification when good labels already exist, as they do here.

41.7 Word embeddings: the bridge to LLMs

From counting words to placing them

TF-IDF treats every word as an unrelated column: "broken" and "damaged" share no similarity at all unless they literally co-occur in the same documents enough to matter. A **word embedding** instead learns a short vector for every word such that words used in similar contexts end up near each other, whether or not they ever appear in the same document. **Word2vec** learns this by training a small network to predict a word from its neighbors (or vice versa) across millions of sentences; the byproduct, the hidden layer's weights, becomes each word's vector.

```
from gensim.models import Word2Vec

sentences = [re.findall(r"[a-z]+", body.lower()) for body in tickets["body"]]
model = Word2Vec(sentences, vector_size=50, window=5, min_count=5, seed=41, workers=1)
print(f"vocabulary size: {len(model.wv):,} words from {len(sentences):,} tickets")
for word in ["broken", "invoice", "order"]:
    neighbors = model.wv.most_similar(word, topn=5)
    print(f"\nwords used most like '{word}':")
    for neighbor, score in neighbors:
        print(f"    {neighbor:<14} {score:.3f}")
```

```
vocabulary size: 284 words from 3,000 tickets
```

```
words used most like 'broken':
```

completely	0.989
received	0.946
right	0.936
down	0.889
is	0.879

```
words used most like 'invoice':
```

shows	0.930
accounts	0.912
amount	0.908
rs	0.894
wrong	0.875

```
words used most like 'order':
```

share	0.615
an	0.612
update	0.607
could	0.601
fr	0.593

Reading it. With only 3,000 short tickets, this is a small, illustrative model, not a production one (real word2vec models train on billions of words). Even so, the neighbors it finds for "broken" (*completely, received, crack*) and for "invoice" (*shows, accounts, amount, wrong*) are words that genuinely keep company with those ideas in this dataset. "Order"'s neighbors are weaker and noisier, because "order" appears in nearly every ticket regardless of topic and so has less distinctive context to learn from.

Seeing the space

```
from sklearn.decomposition import PCA

vocab_words = [
    w
    for w in [
        "broken",
        "crack",
        "damaged",
        "invoice",
        "gst",
        "refund",
        "delivery",
        "dispatch",
        "tracking",
        "order",
        "cancel",
        "address",
        "bulk",
        "pricing",
        "catalog",
    ]
    if w in model.wv
]
vectors = np.array([model.wv[w] for w in vocab_words])
coords = PCA(n_components=2, random_state=41).fit_transform(vectors)
for word, (x, y) in zip(vocab_words, coords):
    print(f"{word:<12} {x:>6.2f} {y:>6.2f}")
```

broken	3.58	-1.98
crack	4.82	-0.86
damaged	1.67	1.94
invoice	-3.64	0.02
gst	-3.35	0.62
refund	2.49	3.11
delivery	-0.58	-2.43
dispatch	-0.74	3.94
tracking	-0.41	2.20
order	-0.97	0.36
cancel	-0.99	-1.20
address	-0.73	-3.69
bulk	0.02	-0.93
pricing	-1.17	-1.09

Fifteen ticket words by their word2vec PCA position



Figure 41.2 – Fifteen ticket words, placed by the first two principal components of their embeddings. Words that mean similar things in this dataset land near each other, without ever being told a category.

Reading it, with Chapter 38's caution firmly in mind. *Broken* and *crack* land close together, as do *invoice* and *gst*, and *dispatch* and *tracking*: word2vec has found real structure using nothing but which words appear near which. It's still a PCA plot of a small model: don't read exact distances, and expect a bigger corpus to sharpen these clusters further.

This is precisely the idea, scaled up enormously and made contextual (so "crate" gets a different vector depending on whether it's near "broken" or near "delivered"), behind the embeddings inside every modern large language model. Chapter 54 picks this up directly: the dot product between two embeddings is the same cosine similarity from section 41.3, and it's the mechanism behind both this chapter's "find similar tickets" and an LLM's attention.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Not deciding on a cleaning pipeline deliberately	Different results every time someone reruns it	Fix and document tokenization, stop words, and stemming/lemmatization choices
Removing stop words before checking the task	Sentiment or negation breaks ("not good" loses "not")	Check what the task needs; some stop words carry meaning
Comparing stemmed and unstemmed vocabularies	Vocabulary size looks inconsistent between runs	Apply the same pipeline to train and any new text
Trusting near-perfect classification scores	99%+ accuracy on messy real-world text	Check for templated or duplicated text; expect real text to score lower

Mistake	Symptom	Fix
Reading a confusion matrix without reading the errors	Missing the pattern in what's misclassified	Read a sample of the actual misclassified documents
Using a lexicon for domain-specific sentiment	"Positive" score on a billing complaint	Train a classifier on your own labeled examples once you have some
Assuming discovered topics match known categories	Confusion when LDA topics "don't make sense"	Cross-tabulate against known labels; report what was actually found
Picking the number of topics arbitrarily	Topics that are too broad or too fragmented	Try several counts; judge by whether a person can name each topic
Treating TF-IDF or word2vec vectors as ground truth similarity	Overconfident "these two tickets are duplicates" claims	Use similarity as a ranking signal, not a certainty
Training word embeddings on too little text	Noisy, unstable neighbor words	Use pretrained embeddings for small corpora, or expect noise
Forgetting to fit the vectorizer only on training data	Leakage of test vocabulary into training	Fit <code>TfidfVectorizer</code> inside a pipeline, same as any other transformer (Chapter 36)

In the real world: the topics nobody had named

In August 2026, Riverstone's support lead, Priya Menon, asks Meera for "a report on what customers are actually complaining about", beyond the five categories the ticketing system forces every agent to pick from.

Meera runs both approaches from this chapter. The classification model into the five official topics works fine and isn't the interesting part; Priya already has that dashboard. The topic model is what she was actually asking for, and Meera is careful with how she presents it, because Chapter 38's lesson applies directly here too: an unsupervised result is a hypothesis, not a finding, until it's checked against something real.

The topic model's most distinct cluster isn't about any product category. It's a cluster of short, curt, action-demanding language: "immediately", "unacceptable", "second time", "escalate". Cutting across Delivery, Billing, and Product Defect tickets alike. Meera pulls fifty of those tickets and reads them by hand (the check no algorithm replaces). They *are* a real pattern: customers who have already contacted support once before about the same order and are writing again.

That's not one of the five topics. It's a **repeat-contact rate**, hiding inside every category. Meera checks it against something the topic model never saw: whether the ticket's `order_id` appears more than once in the ticket log. It does, for the great majority of that cluster.

Her report to Priya doesn't recommend a sixth category in the ticketing system. It recommends a repeat-contact flag, shown to agents the moment a second ticket comes in on the same order, and a weekly count of how many tickets are second contacts, as a service-quality metric in its own right. The topic model didn't answer Priya's question by handing her clean labels; it pointed at a question worth asking with a much simpler, more reliable check.

Tools

- **NLTK 3.10.3** (`pip install nltk`; needs `nltk.download(...)` for `punkt`, `stopwords`, `wordnet`, and `vader_lexicon` on first use): tokenization, stop words, stemming, lemmatization, VADER sentiment.
 - **scikit-learn 1.8.0**: `CountVectorizer`, `TfidfVectorizer`, `MultinomialNB`, `LogisticRegression`, `LatentDirichletAllocation`, `NMF`, `cosine_similarity` — the same classes used in Chapters 36–38, now applied to text.
 - **gensim 4.4.0** (`pip install gensim`): `Word2Vec`.
 - Not used here but worth knowing for production text work: **spaCy** (faster, more modern tokenization and lemmatization, named-entity recognition), **sentence-transformers** (pretrained sentence embeddings, far stronger than a word2vec model trained on 3,000 tickets), and **Hugging Face transformers** (pretrained classifiers and modern language models, Chapter 54).
 - Everything ran on one CPU core, Python 3.12.3, on 18 September 2026.
 - **Companion files**: `companion/generate_riverstone_tickets.py` (seed 20241) builds `companion/tickets/tickets.csv`. Run the chapter's code from `companion/ch41/`. Data spec: `planning/data/riverstone-tickets.md`.
-

The project: classify and summarize Riverstone support tickets by topic

Goal: a working ticket classifier, an honest sentiment comparison, and a topic-discovery pass that finds something the official categories miss.

Option A: your own text. Any collection of short documents with at least one label you can check against: emails, reviews, survey responses.

Option B: Riverstone. The support tickets.

Steps:

1. **Clean deliberately.** Choose and justify your tokenization, stop-word, and stemming/lemmatization settings in two sentences.
2. **Build TF-IDF** and show, by hand, the tf, idf, and tf-idf for two words in one document.
3. **Classify** the topic with Naive Bayes and logistic regression. Report accuracy, macro F1, and the confusion matrix, and read three misclassified examples aloud (on paper) — what do they have in common?
4. **Check for an inflated score** the way section 41.4 did: does the result survive removing an obvious shortcut (IDs, a distinctive template phrase, whatever your data's version is)?
5. **Sentiment, two ways:** score every document with a lexicon and with a classifier trained on your own labels (or a proxy, like a star rating). Compare accuracy and read two disagreements.
6. **Topic model** with LDA or NMF at two or three different numbers of topics. For each, name every topic in five words or fewer, and say whether a person could actually use the result.

7. **Cross-tabulate** discovered topics against any known label and report the honest match, good or bad.
8. **Word vectors**: train a small word2vec model (or load a pretrained one if your corpus is small) and show five word-neighbor examples that make sense and one that doesn't.
9. **Write a one-page summary** for a non-technical reader: what the classifier does, how reliable the sentiment score is, and one thing the topic model found that a fixed category list would have missed.

Stretch goals:

- Add **bigrams** (`ngram_range=(1, 2)`) to the classifier and see whether phrases like "not good" or "second time" change accuracy or the confusion matrix.
- Try **spaCy** for lemmatization and compare its output with NLTK's on five tricky words.
- Use `sentence-transformers` to embed whole tickets (not just words) and compare its nearest-neighbor tickets with section 41.3's TF-IDF ones.
- Fit LDA with 3, 5, 8, and 12 topics and plot a simple coherence or perplexity curve alongside your own judgment of which count gives the most nameable topics.

You've got it when...

- ☐ I can tokenize text and explain what stop-word removal, stemming, and lemmatization each do and don't do.
- ☐ I can build a bag-of-words table and a TF-IDF table by hand for a handful of short documents.
- ☐ I can explain in one sentence why TF-IDF downweights common words.
- ☐ I train a text classifier with the same evaluation habits as any other model: confusion matrix, per-class metrics, and a look at the actual errors.
- ☐ I'm suspicious of near-perfect text classification scores and know what to check.
- ☐ I know a sentiment lexicon needs no training data and no idea what my business considers bad news, and I know when a trained classifier is worth the labelling effort.
- ☐ I can run LDA or NMF, and I check discovered topics against known labels before claiming they mean something.
- ☐ I can explain what a word embedding captures that TF-IDF can't, and I don't over-read a small model's results.
- ☐ I see the line from cosine similarity (Chapter 35) through TF-IDF and word2vec to what an LLM does with text (Chapter 54).

Recap

- Text becomes usable data through **cleaning**: lowercasing, **tokenization**, optional stop-word removal, and **stemming** or **lemmatization**. Every choice is a modeling decision.
 - **Bag of words** counts words per document, ignoring order; **TF-IDF** reweights those counts by how distinctive each word is across the collection, and documents become comparable with **cosine similarity**.
 - **Text classification** reuses Naive Bayes and logistic regression unchanged, fed TF-IDF features; evaluate it with the same confusion-matrix habits as any classifier, and be suspicious of scores that look too good.
 - **Lexicon-based sentiment** (VADER) needs no training data and doesn't know your business; a **trained sentiment classifier** learns your specific language and beats a generic lexicon once you have labels.
 - **Topic modeling** (LDA, NMF) finds word co-occurrence patterns with no labels; check discovered topics against real categories before trusting them, and expect them to find something different, not confirm what you already know.
 - **Word embeddings** place words in space so that similar usage means nearby vectors, extending TF-IDF's simple co-occurrence counting; this idea, scaled up, underlies modern language models.
-

Practice exercises

Code exercises run from `companion/ch41/` after the chapter's code (they use `tickets`, `train`, `test`, `topic_pipeline`, `lr_topic`, `lexicon_pred`, `model` [the word2vec model], and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) Tokenize and lowercase: "Order #4471 hasn't arrived — 3 days late!" List the tokens, then list what remains after removing punctuation and stop words.
2. (*hand*) A word appears in 2 of 10 documents. Using scikit-learn's formula, $\text{idf} = \ln((1 + N) \div (1 + \text{df})) + 1$, compute its idf. Now compute it for a word in 9 of 10 documents. Which word gets more weight in a document where both appear once?
3. (*hand*) Two short reviews: "not good, would not buy again" and "good, would buy again". As bag-of-words with no stop-word removal, are their vectors identical, similar, or very different? What does that reveal about bag-of-words and negation?
4. Which tool fits each job: (a) you have 200 labeled examples of spam and not-spam; (b) you have 50,000 unlabelled product reviews and want to know what people write about; (c) you need a same-day sentiment score with zero labeled data; (d) you want to find tickets similar to one a customer just sent?

Core

5. Rebuild the classifier using **stemmed** text (apply the Porter stemmer to every ticket before vectorizing) instead of raw words. Does accuracy change? Does the vocabulary size?
6. Rebuild the TF-IDF vectorizer **without** removing stop words and **without** limiting `max_features`. Compare vocabulary size and accuracy with the original. Is bigger always better here?
7. For the sentiment task, compute precision and recall **per class** (frustrated, neutral, positive) for both the lexicon rule and the trained classifier. Which class does the lexicon fail hardest on, and why does that match what section 41.4 found?
8. Fit LDA with 3 topics and with 8 topics instead of 5. For each, read the top words of every topic and say in one sentence whether the topics are more or less nameable than at 5.
9. Find the ticket with the **lowest** maximum topic probability from the LDA model (the one LDA is least sure how to categorize). Read it. Does it read as genuinely ambiguous?
10. Using the word2vec model, find the five nearest neighbors of "late" and "refund". Are they more or less coherent than "order"'s neighbors from the chapter? Why might that be?

Stretch

11. Build a classifier using **only bigrams** (`ngram_range=(2, 2)`) instead of single words plus bigrams. Compare accuracy with the original (1, 2) version.
12. Train the word2vec model with `vector_size=10` and with `vector_size=100`, keeping everything else fixed. Compare the neighbors of "broken" for each. What trade-off does vector size control?
13. Combine TF-IDF features with two numeric features (`resolution_hours`, ticket length) using `ColumnTransformer`, and see whether sentiment classification improves.

Think about it

14. A manager sees your topic model's five topics and asks, "why doesn't this match our five support categories?" Answer in three sentences.
15. Your sentiment classifier scores 99% on held-out tickets from the same period it was trained on. Six months later, on new tickets, it performs much worse. Give two explanations rooted in this chapter and Chapter 36.
16. Someone proposes using cosine similarity between TF-IDF vectors to automatically merge "duplicate" support tickets above a threshold. What could go wrong, and what would you check before turning it on?

Key terms

unstructured text · tokenization · stop words · stemming · Porter stemmer · lemmatization · lemma · bag of words · vocabulary · term frequency (TF) · inverse document frequency (IDF) · TF-IDF · cosine similarity (text) · text classification · confusion matrix (multi-class) · macro F1 · sentiment analysis · sentiment lexicon · VADER · polarity score · trained sentiment classifier · topic modeling · Latent

Dirichlet Allocation (LDA) · non-negative matrix factorization (NMF) · document-topic distribution · topic-word distribution · word embedding · word2vec · context window · nearest neighbors (embedding space) · pretrained embeddings · n-gram · bigram

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 38, Unsupervised Learning**, supplied the judgment habits (stability, external checks, "is this real?") this chapter applied to topic models.
- **Chapter 42, Recommender Systems & Ranking**, reuses TF-IDF and cosine similarity directly for content-based recommendations.
- **Chapter 54, Generative AI & Large Language Models**, picks up word embeddings exactly where this chapter leaves them: contextual, much larger, and wired into attention and generation.
- **Chapter 55 and 58** use document embeddings (a whole-ticket version of section 41.7's word vectors) for search and retrieval.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, is exactly what governs the classifiers in this chapter; nothing about text changes how you should evaluate them.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers TF-IDF versus embeddings, bag-of-words limitations, when to use a lexicon versus a trained model, and "how would you find the topics in a pile of customer feedback?"

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Tokens (lowercased): `order`, `#`, `4471`, `has`, `n't`, `arrived`, `-`, `3`, `days`, `late`, `!`. (NLTK's tokenizer splits contractions like "hasn't" into `has` and `n't`.) After removing punctuation, numbers, and stop words: `order`, `arrived`, `days`, `late` — "hasn't" is lost entirely, which is exactly the negation problem section 41.3 flags: the sentence went from "hasn't arrived" (bad) to a bag of words that reads neutral.
2. idf for the word in 2 of 10: $\ln(11 \div 3) + 1 = \ln(3.667) + 1 = 1.299 + 1 = \mathbf{2.299}$. For the word in 9 of 10: $\ln(11 \div 10) + 1 = \ln(1.1) + 1 = 0.095 + 1 = \mathbf{1.095}$. The rarer word gets more than twice the weight of the common one for the same single occurrence — TF-IDF is doing exactly what it's meant to.
3. As plain bag-of-words with no stop-word removal, the two sentences share every single word ("not", "good", "would", "buy", "again") in the same counts — their vectors are **identical**. That's a serious failure: the reviews mean opposite things. Bag-of-words has no concept of word order or of "not"

attaching to the word next to it; fixing this needs either bigrams (so "not good" becomes its own feature, distinct from "good") or a method that reads sequences, like the embeddings in section 41.7 or the models in Chapter 54.

4. (a) **A trained classifier** (Naive Bayes or logistic regression on TF-IDF): you have labels, so use them. (b) **Topic modeling** (LDA or NMF): no labels, and the goal is discovery, not prediction. (c) **A sentiment lexicon** like VADER: zero labeled data and speed are the priority; accept the accuracy trade-off from section 41.4. (d) **TF-IDF plus cosine similarity** (section 41.3): a direct similarity search needs no training at all.

5.

```
from nltk.stem import PorterStemmer

stemmer_ = PorterStemmer()

def stem_text(text):
    return " ".join(stemmer_.stem(w) for w in text.lower().split())

train_stemmed = train["body"].apply(stem_text)
test_stemmed = test["body"].apply(stem_text)
lr_stemmed = topic_pipeline(LogisticRegression(max_iter=1000)).fit(
    train_stemmed, train["topic"]
)
pred_stemmed = lr_stemmed.predict(test_stemmed)
vocab_stemmed = lr_stemmed.named_steps["tfidf"].vocabulary_
print(
    f"stemmed:  accuracy {(pred_stemmed == test['topic']).mean():.3f}  "
    f"vocabulary {len(vocab_stemmed):,}"
)
vocab_original = lr_topic.named_steps["tfidf"].vocabulary_
print(
    f"original:  accuracy {(lr_topic.predict(test['body']) == test['topic']).mean():.3f}  "
    f"vocabulary {len(vocab_original):,}"
)
```

```
stemmed:  accuracy 0.995  vocabulary 1,360
original:  accuracy 0.995  vocabulary 1,331
```

Accuracy barely moves (this data's topics are already easy to separate, so there's little room to improve), but the vocabulary shrinks noticeably, because stemming folds related word forms together. On a harder, more realistic classification problem with less templated language, that shrinkage usually helps more, by letting the model generalize from "deliver" to "delivering" it has seen only one form of in training.

6.

```

from sklearn.feature_extraction.text import TfidfVectorizer as TV

wide_pipeline = Pipeline(
    [("tfidf", TV()), ("model", LogisticRegression(max_iter=1000))]
)
wide_pipeline.fit(train["body"], train["topic"])
wide_pred = wide_pipeline.predict(test["body"])
print(
    f"no stop words, no feature cap: vocabulary "
    f"{len(wide_pipeline.named_steps['tfidf'].vocabulary_):,} "
    f"accuracy {(wide_pred == test['topic']).mean():.3f}"
)
print(
    f"original (stop words removed, max_features=3000): vocabulary "
    f"{len(lr_topic.named_steps['tfidf'].vocabulary_):,} "
    f"accuracy {(lr_topic.predict(test['body']) == test['topic']).mean():.3f}"
)

```

```

no stop words, no feature cap: vocabulary 2,770    accuracy 0.995
original (stop words removed, max_features=3000): vocabulary 1,331    accuracy 0.995

```

The unrestricted vocabulary is several times larger and includes every stop word and every unique order number as its own feature, yet accuracy doesn't improve — those extra features are noise for this task, not signal. Bigger isn't automatically better: more features mean more chances to fit noise and a slower, harder-to-explain model, so `max_features` and stop-word removal are usually worth keeping unless a specific test shows they're costing you accuracy.

7.

```

from sklearn.metrics import precision_score, recall_score

for label in ["frustrated", "neutral", "positive"]:
    lex = lexicon_pred.astype(str) == label
    true = tickets["sentiment"] == label
    print(
        f"{label:<12} lexicon    precision {precision_score(true, lex):.3f}  "
        f"recall {recall_score(true, lex):.3f}"
    )
for label in ["frustrated", "neutral", "positive"]:
    pred = pred_sent == label
    true = test["sentiment"] == label
    print(
        f"{label:<12} classifier precision {precision_score(true, pred):.3f}  "
        f"recall {recall_score(true, pred):.3f}"
    )

```

frustrated	lexicon	precision 0.909	recall 0.771
neutral	lexicon	precision 0.853	recall 0.395
positive	lexicon	precision 0.359	recall 0.978
frustrated	classifier	precision 1.000	recall 0.996
neutral	classifier	precision 0.997	recall 1.000
positive	classifier	precision 1.000	recall 1.000

The lexicon's worst class is **neutral**: its recall is poor because, as section 41.4 showed, polite factual language reads as mildly positive to a general-purpose dictionary, so many truly neutral tickets get called positive instead. The trained classifier's precision and recall are all near 1.0 across every class,

again inflated by the templated data, but the *pattern* — the lexicon struggling specifically on neutral — matches the confusion table exactly and would likely persist on real, less templated text.

8.

```
for k in [3, 8]:
    small_lda = LatentDirichletAllocation(
        n_components=k, random_state=41, max_iter=20
    ).fit(counts)
    print(f"\n--- LDA with {k} topics ---")
    for i, component in enumerate(small_lda.components_):
        top = [words[j] for j in component.argsort()[-8:][::-1]]
        print(f"topic {i}: {' '.join(top)}")
```

```
--- LDA with 3 topics ---
topic 0: order, invoice, team, immediately, dear, gst, copy, accounts
topic 1: order, update, days, delivery, check, thanks, wanted, address
topic 2: order, dear, team, days, just, time, units, called

--- LDA with 8 topics ---
topic 0: order, team, immediately, invoice, dear, gst, copy, accounts
topic 1: board, chopping, order, address, delivery, change, week, godown
topic 2: order, days, called, ago, team, dear, quantity, time
topic 3: order, invoice, does, dear, team, rep, sales, match
topic 4: order, bulk, website, soon, range, place, really, impressed
topic 5: time, just, order, details, gst, terms, payment, enquiring
topic 6: update, order, days, wanted, check, status, share, dispatch
topic 7: days, order, page, tracking, just, updated, checking, reach
```

At 3 topics, General Enquiry and Order Change get folded into the same broad buckets as Delivery and Billing, and topic 2's word list ("order, dear, team, days, just, time, units, called") reads as a vague mix of several real categories rather than one nameable thing. At 8, several topics split into near-duplicate variants of the same Delivery theme (topics 2, 6, and 7 are all "checking on my delayed order" with slightly different words), which is fragmentation rather than genuine new structure, while topic 1 oddly groups "chopping board" with "address" and "godown" purely because those words happened to co-occur. 5, as used in the chapter, isn't obviously "correct" either; the right number is a judgment call, made by reading the topics, not a number a formula hands you.

9.

```
topic_probs = lda.transform(counts)
least_confident = topic_probs.max(axis=1).argmin()
print(f"max topic probability: {topic_probs[least_confident].max():.3f}")
print(f"true topic: {tickets.loc[least_confident, 'topic']}")
print(tickets.loc[least_confident, "body"])
```

```
max topic probability: 0.605
true topic: Product Defect
One of the storage bin units in order 103745 seems to hae a anufacturing defect. Can you advise
next steps?
```

The least-confident ticket (top probability only 0.605) is a genuinely messy one: a typo-heavy Product Defect ticket ("hae a anufacturing defect") whose garbled words give LDA less to grab onto, not one of the explicitly mixed tickets from section 41.4. LDA's uncertainty here reflects the document being genuinely hard to place, whether from mixed content or, as here, from noisy text eroding the very words the topic model relies on — the same conclusion a human skimming it quickly might reach.

10.

```
for word in ["late", "refund"]:
    if word in model.wv:
        print(f"\nnighbors of '{word}':")
        for neighbor, score in model.wv.most_similar(word, topn=5):
            print(f"    {neighbor:<14} {score:.3f}")
```

```
neighbors of 'late':
unhappy      0.964
courier      0.873
extremely    0.816
no           0.809
update       0.765

neighbors of 'refund':
account      0.933
unacceptable 0.907
immediately  0.906
please       0.836
unusable     0.835
```

"Late" and "refund" each appear in a narrower set of contexts than the near-universal "order", so their neighbor lists are more topically coherent: "late" surrounds itself with delivery-frustration words (*unhappy*, *courier*, *extremely*), and "refund" with billing-and-defect words (*unacceptable*, *immediately*, *unusable*). "Order" appears in almost every template regardless of topic, giving it a broad, low-signal context that produces noisier, less thematic neighbors, exactly as the chapter noted.

11.

```
bigram_only = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            stop_words="english", max_features=3000, ngram_range=(2, 2), min_df=2
        ),
    ),
    ("model", LogisticRegression(max_iter=1000)),
])
bigram_only.fit(train["body"], train["topic"])
print(
    f"bigrams only: accuracy "
    f"{(bigram_only.predict(test['body']) == test['topic']).mean():.3f}"
)
print(
    f"unigrams + bigrams (original): accuracy "
    f"{(lr_topic.predict(test['body']) == test['topic']).mean():.3f}"
)
```

```
bigrams only: accuracy 0.995
unigrams + bigrams (original): accuracy 0.995
```

Bigrams alone classify just as well here (99.5%, identical to the original), because this data's fixed templates produce very consistent two-word phrases ("invoice for", "does not") that carry as much signal as single words do in this particular dataset. On less templated real text, single distinctive words are usually cheaper to estimate reliably and just as informative, which is why the standard choice is unigrams plus bigrams together rather than either alone.

12.

```
for size in [10, 100]:
    small_model = Word2Vec(
        sentences, vector_size=size, window=5, min_count=5, seed=41, workers=1
    )
    if "broken" in small_model.wv:
        print(f"\nvector_size={size}, neighbors of 'broken':")
        for neighbor, score in small_model.wv.most_similar("broken", topn=5):
            print(f"    {neighbor:<14} {score:.3f}")
```

```
vector_size=10, neighbors of 'broken':
completely      0.990
right           0.955
received        0.951
down            0.935
crack           0.888

vector_size=100, neighbors of 'broken':
completely      0.989
received        0.952
right           0.935
down            0.890
is              0.878
```

Here the two sizes give nearly the same top neighbors for "broken" (completely, received, right, down, crack all appear in both lists, just reordered), because with only 3,000 short tickets there isn't enough data to make good use of 100 dimensions — most of the extra room in the larger model goes unused rather than capturing genuine nuance. A smaller vector size forces meaning into fewer numbers and can blur real distinctions on a large corpus; a larger size needs more data to fill its extra room usefully. This is a case where the corpus size, not the vector size, is the real bottleneck (section 41.7's caution).

13.

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

tickets["body_length"] = tickets["body"].str.len()
train2 = tickets.loc[train.index]
test2 = tickets.loc[test.index]
combined = ColumnTransformer(
    [
        (
            "text",
            TfidfVectorizer(
                stop_words="english", max_features=3000, ngram_range=(1, 2), min_df=2
            ),
            "body",
        ),
        ("numeric", StandardScaler(), ["resolution_hours", "body_length"]),
    ]
)
combined_pipeline = Pipeline(
    [("features", combined), ("model", LogisticRegression(max_iter=1000))]
)
combined_pipeline.fit(train2, train2["sentiment"])
combined_pred = combined_pipeline.predict(test2)
print(
    f"text + numeric features: accuracy {(combined_pred == test2['sentiment']).mean():.3f}"
)
print(
    f"text only (section 41.4): accuracy {(pred_sent == test['sentiment']).mean():.3f}"
)

```

```

text + numeric features: accuracy 0.999
text only (section 41.4): accuracy 0.999

```

Adding `resolution_hours` and ticket length changes nothing here (99.9% either way), because the generator ties sentiment to word choice directly and only loosely to resolution time; the text already carries essentially all the signal. On real data, response-time and length features often do add value for sentiment or urgency prediction, so this combination pattern (mixing a `TfidfVectorizer` column with numeric columns inside one `ColumnTransformer`) is worth having ready even when, as here, it doesn't move the needle.

14. "Your five categories were designed by people, for a purpose — routing tickets to the right team — and they mix several different things people care about: which product, which stage of the order, and how urgent the tone is. The topic model only sees which words appear together, so it found a pattern in *tone* (urgent, repeat-contact language) that cuts across your categories rather than matching them. That's not the model failing; it's the model finding a different, real pattern your category list wasn't built to capture."

15. First, **the language itself may have drifted** (Chapter 36's idea of drift, applied to text): new products, new complaint types, or seasonal issues introduce words and phrasings the classifier never saw in training, and a model trained on last year's vocabulary has no way to handle them. Second, the near-99% training-period score may itself have been inflated by **something specific to that period** the model quietly learned — a promotion that generated a wave of similarly worded tickets, or a support-team habit of using stock phrases that later changed — rather than the underlying sentiment

signal being that clean everywhere. Both point to the same fix: retrain regularly on recent labeled data and monitor accuracy on a rolling basis, exactly as Chapter 39's monitoring section recommends for any deployed model.

16. The main risk is treating **high textual similarity as proof the tickets are about the same issue**, when two customers can describe unrelated problems in nearly identical boilerplate language (this chapter's mini-example ticket 0 and its "duplicates" are literally different customers, different order numbers, different actual invoices — similar only because they used the same template). Merging on text alone could hide a second customer's genuine, distinct complaint inside a closed ticket. Before turning it on: check that a matched pair also shares something that should be shared for a true duplicate (same customer, same order ID, tickets close together in time), read a sample of pairs above the proposed threshold by hand, and make the system suggest a merge for a human to confirm rather than merging automatically.